
SCHOOL OF ENGINEERING AND TECHNOLOGY

FINAL ASSESSMENT FOR BSC (HONS) IN COMPUTER SCIENCE; BSC (HONS) INFORMATION TECHNOLOGY; BACHELOR OF SOFTWARE ENGINEERING (HONS); BSC (HONS) INFORMATION SYSTEMS (DATA ANALYTICS)

YEAR 2; ACADEMIC SESSION SEPTEMBER 2026

SEG2102 DATABASE MANAGEMENT SYSTEMS

DUE DATE: BY END OF THE 3 JANUARY (SUNDAY), 2026

INSTRUCTIONS

- This final assessment contributes **30% to final grade**.
- This **three-member (3-member) group assignment** is primarily for Course Learning Outcomes 1 and 3 - Examine the advanced concept of different database models concerning their design, strengths, and weaknesses from the aspects of models' design, data integrity, database scalability, and access control.
- **Submission:** Submit a **single MS Word file** as your report. This assessment requires GROUP submission through the **TURNITIN** system, meaning GROUP LEADER must submit their own report based on the group work via **TURNITIN**.
- **Report Format:** Prepare a report using the **IEEE template**.
- **Detailed Requirements:** For specific details about the assignment, please refer to pages 2-3.
- **Plagiarism Policy:** Turnitin report must indicate **less than 20% similarity**, including AI-generated content.

IMPORTANT

The University requires students to adhere to submission deadlines for any form of assessment. Penalties are applied in relation to unauthorized late submission of work.

Academic Honesty Acknowledgement

"We **Isabelle Yau Li Shuang, Leow Yan Yi, Peggy Liow En Ki** verify that this paper contains entirely our own work. We have not consulted with any outside person other than what was specified in the assignment requirements. Further, we have not copied or inadvertently copied ideas, sentences, or paragraphs from another student. We realize the penalties (*refer to the student handbook of undergraduate programme*) for any kind of copying or collaboration on any assignment."

Signature: *Isabelle Yau Li Shuang, Leow Yan Yi, Peggy Liow En Ki*

Student ID: 23021165, 23020324, 23020852

Name: Isabelle Yau Li Shuang, Leow Yan Yi, Peggy Liow En Ki

Date: 2nd January 2026

Data Storage, Indexing and Retrieval

Isabelle Li Shuang Yau
School of Computing and Artificial
Intelligence
Sunway University
Selangor, Malaysia
23021165@imail.sunway.edu.my

Peggy En Ki Liow
School of Computing and Artificial
Intelligence
Sunway University
Selangor, Malaysia
23020852@imail.sunway.edu.my

Yan Yi Leow
School of Computing and Artificial
Intelligence
Sunway University
Selangor, Malaysia
23020324@imail.sunway.edu.my

Nowadays, data storage, indexing, and retrieval are essential daily operations performed by businesses to manage their transactional data. Businesses often face challenges related to inefficient data retrieval methods and degraded query performance as the volume of data increases. Hence, this paper aims to evaluate the performance of different indexing methods, such as B-tree and hash indexing, in comparison with a non-indexed table within MySQL and NoSQL relational database environment, with a focus on their impact on query execution performance and storage overhead. The report also aims to recommend the optimal indexing strategy for retail transaction data which maximizes the retrieval performance while minimizing storage overhead. A quantitative experimental methodology was adopted, comparing the three different scenarios using point lookup, range, sorting, and aggregation queries in both MySQL and NoSQL environments. The results show that indexing methods outperform tables without indexing in terms of query execution performance. Overall, B-tree indexing is the best indexing technique compared to hash indexing as it consistently demonstrates low execution time across different query types with minimal storage overhead. In short, the findings suggest that B-tree indexing should be adopted by retail databases, as it optimizes the query performance while maintaining the storage efficiency of the database.

Keywords—*B-tree Index, Hash Index, MySQL, Query Performance, Storage Overhead*

I. INTRODUCTION

A. General Background

The ability of efficient data handling is a crucial factor to a business's success. Database Management Systems (DBMS) serves as the backbone of business operations by facilitating data storage and daily transactions. The performance of the database becomes increasingly important as the data amounts grow with the business.

One of the most common challenges in large-scale business datasets is the degradation of query performance. As the volume of data increases, simple data retrieval operations are prone to slow down more significantly, thus creating query latency bottlenecks. To mitigate the issue, indexing techniques are used to allow the database managements systems to locate data without the need to fully scan the entire database.

This assignment report focuses on the performance optimisation of a retail dataset using MySQL and MongoDB. It explores the impact of difference indexing strategies on data retrieval speeds. This report evaluates how B-Tree and Hash indexing methods compare against non-indexed storage in a relational database environment.

B. Problem Statement

Inefficient data retrieval methods in large-scale databases lead to increased query latency and higher CPU usage, thus leading to poor system performance. In the case of the Super

Store dataset, queries that are not indexed will automatically result in a full table scan. This forces the database to inspect every single row to find the match, which is not scalable and computationally inefficient.

Choosing the most suitable index type is important to prevent excessive storage overheads without delivering meaningful improvements in query performance. B-Tree indexes are appropriate for a variety of analytical processes since they provide logarithmic time complexity and facilitate range-based queries. However, range searches are not supported by hash indexes, which offer constant-time performance for exact match queries. [1]. The performance implication of complex operations such as multi-attribute filtering, sorting large result sets, and aggregations across these different platforms remain under-explored.

The main issue addressed in this assignment is the lack of empirical information regarding the selection of indexing algorithms for typical retail transaction data. This often leads to poorly optimised database configurations that consume additional storage resources while failing to achieve optimal performance gains.

C. Significance of the Problem

Improving operational effectiveness and user experience requires addressing the trade-off between retrieval speed and storage cost. In an e-commerce environment, high query latency affects webpage loads. Research indicates that minor increases in latency can lead to significant user dissatisfaction and abandonment of sales platforms [2]. Ensuring sub-second retrieval ties is essential for maintaining customer retention.

Furthermore, resolving the problem maximises available resources. Financial costs are associated with database storage and processing power. Organisations can minimise their hardware footprint by determining the most effective indexing approach. This assignment offers a quantitative foundation for balancing the advantages of faster queries against the cost of index storage, providing insightful information for creating scalable retail systems.

D. Objectives

The primary objective of this assignment is to evaluate and compare the performance of different indexing methods within the MySQL and MongoDB environments.

1. To implement and configure the distinct indexing scenarios on the dataset: No Index, B-Tree Index, and Hash Index within both MySQL and MongoDB environments.
2. To measure and analyse the data retrieval speed for Point Lookups, Range Queries, Composite Filtering, Sorting, and Aggregation under these different configurations.
3. To assess the data storage cost by calculating the disk space consumed by the index files.

4. To recommend the optimal indexing strategy for retail transaction data that maximises retrieval performance while minimising storage overhead.

E. Scope of the Report

This report investigates two database management systems: MySQL 8.0 and MongoDB v8.2.3 (NoSQL). This experimental analysis uses the SuperStoreOrders dataset, which contains 51,290 records and 21 attributes, on both platforms to compare relational and document-oriented indexing strategies.

The scope of the comparison is defined below:

1. Database Engine:

The study utilises the MySQL InnoDB engine for both the non-indexed baseline and B-Tree indexing. The MEMORY engine is used for Hash Indexing, as MySQL requires in-memory storage to support hash index structures. For NoSQL, MongoDB is utilised to evaluate B-Tree and Hash Indexing on document collections.

2. Indexing Strategies:

The evaluation is limited to three distinct scenarios: non-indexed, B-Tree indexing, and Hash indexing.

3. Query Types:

The performance analysis focuses on five specific retrieval patterns: Point Lookups, Range Queries, Composite Filtering, Sorting, and Aggregation.

4. Performance metrics:

The comparison is based on two quantitative measures: Query Latency and Storage Overhead. Query latency measures data retrieval speed, while storage overhead captures the space required to maintain index structure.

II. LITERATURE REVIEW

The capability of an indexing database has a significant effect on the efficiency of retrieving and storage utilization in a database management system (DBMS). It may affect the speed of returning results and the overall performance of a database. In this century, the development and execution of an efficient indexing strategy to support rapid querying and scalable access to data is more important, along with the continuous production and expanded quantity. In this case, the most often used indexing techniques are B-Tree and Hash indexing, which are designed to handle different scenarios and workloads. B-tree indexes are used to enhance the range-based and ordered retrieval queries, while the hash indexes are used to optimize the exact-match lookups. Thus, full table scans that may lead to poor performance and latency are required in a database that does not adopt index techniques. Therefore, this literature review session will discuss studies that are currently available and observed indexing techniques, including no-index, hash, and B-tree methods. The objective of this session is to provide an overview, outline important findings, and performance characteristics of these approaches. Besides, it may also discuss and identify the limitations based on the studies to conduct experimental analysis in this study.

A. Review of Articles

First and foremost, Nakazono et al. [3] investigate the performance of traditional B+- tree indexes on transactional

database systems. It states that there are limitations when handling point queries and range queries. Therefore, it proposes a hybrid index named Griffin. It consists of a hash table that can support $O(1)$ point operations and a B+- tree index that supports range operations. It also helps to avoid the circumstances of phantoms by using precision locking techniques without rescanning for each operation. The key finding of this paper is the 3.1x higher throughput compared to other alternative solutions that are used for point queries. Meanwhile, it also has 5.4x higher throughput in terms of mixed point-range workloads. Therefore, it shows the effectiveness of the adoption of hybrid index techniques.

There are also various studies that explore the limitations and ways to enhance the memory and Cache Performance of traditional B- tree, including the cache oblivious approach to creating an efficient structure. A new index structure named CORoBTS has been proposed in Ondráček and Mička [4]. This index structure may help to enhance the memory locality and reduce the complexity of I/O by combining van Emde Boas memory layout and packed memory arrays. Therefore, it may help to enhance the efficiency of search and update operations and maintain the linear space utilization. It also points out that there are many advantages, while the indexing structure considers the performance effects of Memory Hierarchy Awareness on the tree's performance.

An et al. [5] proposed the optimized version of the Marlin Distributed B+ tree, which helps to address the limitation of high synchronization overhead and tail latency on disaggregated memory. In this case, an RDMA – IDU friendly concurrency algorithm, a ternary state node lock, and compression of the critical path used for write operations have been utilized in this optimized version. Besides, the design of Marlin's Leaf nodes, which are used to separate data items from the index entries, helps to avoid the locking conflicts that may be observed while updating concurrently. Therefore, the key findings of this paper point out that Marlin has provided a better way for high-performance indexing in distributed memory systems by comparing with the B+- Trees structure.

Chang and Ma [6] highlight the difficulty of optimizing database query responses on Embedded Avionics, especially the requirement for fast retrieval of navigation. Therefore, they investigate and benchmark various index technologies, including B, B+, CSB+, and T trees. The measurement for comparison includes equal value, range, and multidimensional queries for efficiency in terms of space and time. Besides, it also evaluates R and KD Trees in terms of their capability for proximity queries. As a result, it states that the B+ Tree has the best performance on single-column queries, although R and KD trees are able to process multi-dimensional and space-related queries. Hence, it shows the importance of adopting a suitable tree structure based on the type of query to acquire efficiency.

Al Khaldy et al. [7] point out various limitations of traditional B+ Tree and Bitmap indexes. For example, greater storage overheads and insufficient management of duplicate keys. These limitations may lead to a reduction in efficiency and performance. Therefore, these systems may not be suitable for large databases that require scalability. In this case, this paper proposed a primary index that may create direct links between duplicate entries by associating with sibling pointer links. This may also help to reduce unnecessary duplicate lookups and improve the performance of the system.

As a key finding of this paper, an increase of 33.5% per query in performance and 25% reduction of database storage were observed while conducting the performance tests using a maximum of 10,000,000 records of the MySQL database. Hence, it also proves the scalable and storage-efficient enhancement to indexing.

Pujiono et al. [8] investigate the comparison of binary search and interpolation search in B-tree indexes. It points out the significant difference in their performance while querying B-Tree nodes. In this case, interpolation search has better performance compared to binary search, while dealing with key values that exhibit patterns, although binary search exists as a classic method to access B-tree nodes. This study points out that the efficiency of the resulting query may be affected significantly by the algorithm used in the B-tree node to search for a specific item, especially when the data distributions are skewed or non-uniform. Therefore, this study has pointed out how the search strategy can influence overall B-Tree performance and also other optimisation areas in the B-tree index method.

Schmeißer et al. [9] investigates B+-Tree and other alternative index structures while processing large out-of-core data on Graphics Processing Units (GPUs). These GPUs have been linked to each other by using high-speed network connections. It also shows that the connectivity bottlenecks may affect the performance of data transfer while using traditional indexing structures, including INLJs, and the adoption of hash join techniques may help to address this situation. Therefore, an example of a windowed partitioning optimization technique has been proposed in the study, which may help to provide two to ten times faster results than baseline methods. Hence, the key finding of this study is the importance of hardware-aware index design and index optimization towards the ability of query processing.

Aminuddin et al. [10] investigate the way to develop a more efficient method to manage the database, especially by adopting the B-tree indexing technique to obtain efficient data management. It shows how the B-tree structure can balance the data and provide a faster method at a logarithmic speed for searching, inserting, and deleting as the dataset expands. This study also verified that the B-Tree structure may significantly enhance the query processing speed and the stability of the structure by investigating various empirical studies from journals and electronic databases. Therefore, it shows that the B-tree indexing technique should be adopted in database indexing since the next generation of database management systems should be anticipated in the future.

Tao et al. [11] proposed Pea Hash to address the limitation of the traditional hash table index structure. It points out that Pea Hash obtained adaptability and extendibility to afford more efficient point lookup performance and continuous support for the changes of data distribution. It is a methodology that may use adaptive hashing strategies and an extendible bucket management system to reduce the amount of collisions and cost while using a traditional hash indexing method. The key finding of this paper shows that Pea Hash has significantly better performance compared to the traditional hash index. It also obtains the ability to adapt to the changes in data distribution and create high-performance key-based access to the specific database.

Jin et al. [12] have developed a storage engine named CompassDB, which is a combination of a Two-tier

Perfect Hash Table (TPH) and a key-value pair-based data structure. The adoption of TPH has avoided the existence of overflow buckets with $O(1)$ lookup operations that have a more compact index than conventional LSM trees, which may also lead to a reduction of read or write amplification. The result from this paper shows that the adoption of CompassDB may increase the throughput, average, and tail latency significantly by comparing with RocksDB and PebblesDB. Therefore, the key finding from this paper states that the utilization of optimal hashing may lead to high-performance access to data.

Wang et al. [13] proposed a new solution that focused on the high insertion overheads from the management of dynamic hash indexes with a dataset that is constantly changing and costly rehashes. The PDPH that this paper proposed has combined batches with the adoption of static and dynamic perfect hashing techniques. As a result of using these techniques, the efficiency has been increased, and the frequency and amplitude of rehashes required for any given dataset have been reduced. It also shows that the utilization of PDPH can achieve 2.32x higher throughput compared to traditional hash indexing methods, while providing non-collision access to data. This study also states that many of the optimization methods are related to hash-based indexes and B-tree storage formats. Therefore, it can be used in systems that adopt both B-trees and hash indexes.

Li et al. [14] investigate how to store and map the variable-length data blocks in the deduplication system efficiently. In this paper, a user-mode block device is implemented by using the NBD protocol, and the Rabin Rolling Fingerprint Algorithm is also utilized for partitioning of the data blocks. Meanwhile, it also adopts two indexing methods, which are the hash index and the B+ Tree index. As a key finding result from this paper, the hash indexing method provides better read and write performance under a less metadata-intensive situation compared to the B+ Tree index. Although a hash index has better performance in this domain, it requires more storage space to deal with hash collisions. Therefore, it states that the trade-offs are associated with optimizing storage capacity.

Altahat et al. [15] provided various solutions to address challenges in secure cloud data management. For example, slow retrieval, high encryption overhead, and inefficient integrity verification of large datasets. The methods that are proposed in this paper have integrated B-Tree Indexing to access files faster, Enhanced Merkle Hash Trees (EMHT) to provide scalable verification of file integrity, and Proof of Retrieval to ensure the availability of retrieval of files. The result of the research is the key finding, which highlights that there are various improvements made after enhancing the speed of encryption processes, including 35% improvement in verified integrity checks, 75% time reduction on updating, and a 38% of reduction in overall memory adoption. Therefore, it verified that the method in this paper provides scalability and reliability for the challenges of secure cloud data management.

Li et al. [16] proposed a solution to address the difficulty of optimizing indexing throughput on disk-based databases. The solution proposed enables traditional index structures such as B- Trees to process current workloads more efficiently. This paper evaluates the performance of the proposed updatable learned index and B-Trees based on the disk-based database benchmarks. As a result of this research,

it shows that the B-Tree structure provides better overall robustness, lower update cost, and consistent reliability, although the learned indexes proposed provide better performance in some specific environments. Therefore, these advantages make B-Trees structure a better choice for indexing in production database systems.

Cha et al. [17] point out various limitations of the traditional B+tree indexing structure in terms of workloads generated from time-series workloads. It is because of the poor scalability due to the large amounts of contention created from continuously increasing key values. Therefore, they proposed a new hybrid indexing scheme that combines B+tree nodes and hash tree leaves, named Blink-hash. This method provides higher throughput while maintaining the ability to conduct range queries on the dataset when inserting records. This paper optimizes the splits of the leaf nodes by adopting leaf nodes and a median approximation approach. It also uses a concept called lazy splitting in the method. As a result of this paper, the Blink-hash has achieved a maximum of 91 times improvement on insert performance, and it also obtained comparable to or better performances during scanning, compared with traditional B+-tree indexing structures. Therefore, this paper points out that a conscious decision on sacrificing speed is required while using Blink-hash indexing methods, to ensure a correct indexing structure and be suitable for data that is stored inside.

Lyu and Graefe [18] investigate the performance of join queries by using indexed structures. It points out that although standard indexes may help to maintain the consolidation of data efficiently, they may reduce the speed of retrieval. Besides, the materialized join views that may help to return data rapidly, also require a high maintenance cost due to frequent updates. Thus, this study mentioned a new method that may support all types of join operations, with a moderate maintenance cost, named the merged index. This method is a combination of the sorts of base table indexes with traditional B-tree and log-structured merge (LSM) forest organization. As a final result of this study, the merged index is able to provide a similar ability compared with materialized views, but provides approximately the same efficiency in the maintenance of traditional indexes.

According to Wu et al. [19], much manual effort is required to determine the type and amount of indexes that should be used within a database for workload selection and tuning. Thus, a new solution named Neural Index Search (NIS) has been proposed in this paper. This solution uses reinforcement learning techniques and a neural network to provide an automated mechanism for constructing and

configuring hybrid index architectures based on the query workload of a dataset. The NIS framework adopts ordered and unordered blocks, and learn through analyzing the patterns of queries. Therefore, it may help to provide an efficient structure for the specific database. As a result of this paper, it states that the produced indexes of NIS obtain better performance than traditional memory-based indexing methods. Thus, it also shows that optimization of an index that is based on learning may significantly enhance the performance of queries and also reduce the requirement for storage.

Gao et al. [20] evaluate the methods that are used for compressing B-tree indexes for storing data in databases. It evaluates its performance through various indicators, including time, space, and performance characteristics. There are also seven commonly used B-tree compression algorithms that have been adopted by this paper to verify the results. As a key finding of this paper, the method of combining Head and Tail compressively has provided a consistently better performance on search and insert queries, compared to uncompressed B-tree and many newer algorithms. Thus, this finding may help to understand how performance and storage could be affected by the compression of indexed data.

Lee et al. [21] show that the negative effect on B-Tree Node Space Utilization may lead to a significant increase in Write Amplification (W/A) and unsatisfactory performance in database systems that use Flash-based Storage. Thus, this paper has proposed two optimization methods that may be used to free space for every index and redistribute the data to compress B-Tree Nodes and reduce the amount of unimportant splits in B-Tree Nodes. By investigating the evaluation results from Experiments performed on MySQL or InnoDB databases under OLTP Workloads, it shows a maximum 40% of reduction in storage needed and nearly 200% of increase in transaction throughput. Therefore, as a key finding of this paper, it points out that the design and adoption of B-tree indexes that may enable an efficient retrieval process from a database are important.

Lan et al. [22] evaluate whether the efficiency of updated learning-based index systems is the same as the traditional B-tree data structure in online relational DBMS environments, including ALEX, PGM, and LIPP. It also compares both of them based on different workload types and data sets. As a result of this paper, it shows that the B-tree has the best results among the three designs based on their performance, storage use, and finding speed.

B. Comparison Table

TABLE I. COMPARISON TABLE OF LITERATURE REVIEW

Reference	Problem Statement	Algorithm/Methodologies	Key Findings
Nakazono et al. (2024) [3]	The performance limitations of B+- Tree indexes in terms of point queries and phantom avoidance on transactional database	Griffin Hybrid Index - Use hash table to conduct exact lookups - Use B+- Tree on range queries - Prevent phantoms without rescanning by locking precision	- For point-heavy workloads, it can reach maximum 3.1x higher throughput. - For mixed workloads, it can reach 5.4x improvement. - Prove the efficiency of hybrid indexing.
Ondráček & Mička (2024) [4]	The inefficiency of cache and poor memory locality	CORoBTS	- Reduced cache misses - Improve the efficiency of I/O

	observed in large in-memory B-Tree structures	• Store unaware B-Tree layout • Combine van Emde Boas memory layout • Packed memory arrays	
An et al. (2023) [5]	The high synchronization overhead and tail latency in concurrent B+-Tree operations on disaggregated memory	Marlin B+-Tree • RDMA-friendly concurrency control • Ternary-state node locking • Write-path compression	Significant improvement on writes throughput and latency
Chang & Ma (2024) [6]	Inefficient query performance on embedded avionics databases	Comparative Evaluation • B-Tree, B+-Tree, CSB+-Tree, T-Tree • R-Tree and KD-Tree for spatial queries	B+-Tree performed best for single-column queries while R-Tree better in spatial and multidimensional searches
Al Khaldy et al. (2025) [7]	Storage overhead and inefficient duplicate-key handling in B+-Tree indexes	Sibling Pointer Indexing • Direct linking of duplicate keys • Reduced redundant lookups	Achieved 33.5% faster queries and 25% lower storage overhead
Pujiono et al. (2022) [8]	Inefficiency of binary search within B-Tree node traversal	Search Algorithm Comparison • Binary Search • Interpolation Search	Node-level search performance for skewed data distributions has been improved using interpolation search
Schmeißer et al. (2025) [9]	Poor performance of B+-Tree joins on GPU-based systems	GPU-aware Indexing • B+-Tree indexing • Windowed partitioning optimization	Achieved 3–10× throughput improvement over baseline index nested-loop joins
Aminuddin et al. (2020) [10]	Require efficient data retrieval in large relational databases	• Balanced tree structure • Logarithmic search, insert, delete	Demonstrated consistent $O(\log n)$ performance and improved query efficiency
Tao et al. (2023) [11]	Collision overhead and lack of adaptability in traditional hash indexes	Pea Hash • Extendible adaptive hashing • Dynamic bucket management	Improved point lookup performance and adaptability of changing data distributions
Jiang et al. (2024) [12]	Read or write amplification and latency in LSM-tree-based key-value stores	CompassDB • Two-tier Perfect Hash Table (TPH) • $O(1)$ lookup design	Achieved lower latency and higher throughput compared to RocksDB and PebblesDB
Zhang et al. (2023) [13]	Inefficient retrieval in encrypted speech databases	Hybrid Indexing • Unsupervised hashing • Dynamic B+-Tree indexing	Improved encrypted retrieval efficiency while maintaining scalability
Li et al. (2023) [14]	Inefficiency in block-level deduplication indexing	Index Comparison • Hash Index • B+-Tree Index	Hash index provided faster read or write performance with lower metadata overhead
Altahat et al. (2025) [15]	Slow retrieval and integrity verification in cloud storage systems	Hybrid Secure Index • B-Tree indexing • Enhanced Merkle Hash Tree (EMHT)	Achieved 35% faster integrity verification and 38% lower memory usage
Li et al. (2023) [16]	Limitations of traditional B-Trees in disk-resident DBMS workloads	Learned Index Evaluation • Comparison with B-Tree	B-Trees remained more stable and reliable across varied workloads
Cha et al. (2023) [17]	Insert bottlenecks in B+-Trees for time-series workloads	Blink-hash Hybrid Index • B+-Tree internal nodes • Hash-based leaf nodes	Achieved maximum $91\times$ higher insertion throughput with similar range query performance

Lyu & Graefe (2025) [18]	Trade-off between fast joins and high maintenance overhead	Merged Join Index B-Tree / LSM-based sorted indexes	Query performance comparable to materialized views with lower maintenance cost
Wu et al. (2019) [19]	Manual and inefficient index selection for dynamic workloads	Neural Index Search (NIS) - Reinforcement learning - Hybrid B-Tree and hash blocks	Automatically generated indexes outperform manual designs
Gao et al. (2024) [20]	Space overhead and performance trade-offs in B-Tree compression	Compression Evaluation - Head+Tail compression 7 compression methods	Head+Tail compression improved both search and insert efficiency
Lee et al. (2023) [21]	Low space utilization of B-Tree nodes on flash storage	Node Compaction Techniques - Free-space reservation - Data redistribution	Reduced storage by up to 40% and nearly doubled throughput
Lan et al. (2023) [22]	Uncertainty whether learned indexes outperform B-Trees	Empirical Evaluation Learned indexes vs. B-Tree	Observed that B-Tree as a strong and reliable baseline

C. Literature Review Findings, Gaps, Opportunities

There are various methodologies that may enhance the efficiency of the Database indexing mentioned in the

previous session, while hashing and B-Tree indexing are the two strategies that most researchers have used. In this case, previous studies point out the excellent point query performance and $O(1)$ lookup speed provided by hash indexes [3][11][12][13]. Besides, some papers also mentioned that the adoption of an adaptive or extendible hashing method will help to reduce the amount of collisions and effort of work while inserting the hash index. Tao et al. [11], Marlin [5], and Altahat et al. [15] also show that the most common indexing strategies used for range-query performance, ordered traversal, and consistent update operations are B-Tree and B+-Tree structures. Moreover, Ondráček and Mička [4] and Gao et al. [20] also mentioned memory-aware optimization techniques that can be used on B-Trees to improve search speed and storage utilization, including cache-oblivious layouts and compression. There are also some hybrid designs that combine hashing and B-Tree techniques, which have been mentioned in Li et al. [14] and Cha et al. [17]. They point out that this combination may help to enhance the throughput of insertions while maintaining sufficient range query performance by adopting the advantages of both of them.

On the other side, there are also some uninvestigated gaps that exist in DBMS. Most of the comparative research based on the performance of hash, B-Tree, and hybrid index types in real-time workloads is focused on the trade-offs between insertion speed, query latency, and memory utilization. In this case, hash indexes stand out from other methods, since they perform better than them. On the other side, hash indexes also have limitations since there is limited information available about their performance on range and multidimensional queries. Therefore, it states that there is an opportunity to design an index that is able to adapt and switch indexing strategies intelligently.

Last but not least, there are future works that should be adopted to address these gaps, including systematic testing and benchmarking of the various index types against multiple workloads and work in different storage architectures. Besides, it also required research that may focus on adaptive or memory-aware hybrid index strategies

to improve the efficiency of query processing while providing the benefits of throughput, storage, and query performance.

D. Summary of Literature Review

Various types of indexing methodologies have been proposed in the literature review, with the hash and B-Tree methods as the focused methods. In this case, Griffin [3] and Pea Hash [11] state that the combination of hash and B-Tree indexes may help to enhance the performance of point queries and efficiency on range queries. There are some improvement techniques conducted on B-Tree, including Cache-Oblivious Layouts [4], Distributed Write Optimized Structures [5], and Compression Techniques [20][21]. These techniques may help to enhance memory efficiency, concurrent access, and throughput. Besides, there are various studies that state that these techniques are allowed to enable under specific environments. For example, Embedded Systems [6], sparse GPU-based Large Scale Data Processing [9], and Secure Cloud Storage [15]. Studies that are within five years have concentrated less on the comparison of non-index with either B-Tree or Hash Indexing. Hence, there is limited information on the comparison of non-indexed tables and hash or B-Tree indexed tables. Based on the work presented in this study, a comparison between a non-indexed table, a hash, and a B-Tree indexed table will be conducted. It will compare the performance, storage efficiency, and scalability of each methodology to measure how well they are able to work in a real-world transaction-based database application.

III. RESEARCH METHODOLOGY

A. Introduction to the Methodology

This section outlines the systematic approach used to evaluate and compare the performance of distinct database indexing strategies within a relational database environment (MySQL) and a NoSQL document store (MongoDB). This methodology's main goal is to examine the trade-offs between storage overhead and data retrieval performance when using B-Tree and Hash indexing algorithms on a retail transaction dataset.

The methodology includes data preprocessing, experimental environment setup, and thorough performance benchmarking. This structure ensures that the findings

address the gap identified in the literature regarding the direct comparison of non-indexed, B-Tree, and Hash-indexed tables across SQL and NoSQL platforms in transactional workloads.

B. Research Design

A quantitative, comparative experimental approach was used in the study. It focuses on evaluating the three distinct database configurations: No index, B-Tree Index, and Hash Index. The goal of the study is to quantify the effects of indexing techniques on query performance and system resource usage.

C. Steps Involved in the Methodology

- **Identification of Research Objectives**

The primary objective of this methodology is to evaluate and determine the best indexing approach for retail datasets across relational and non-relational systems. The goal of the study is to quantify the retrieval speed for five distinct retrieval patterns: point lookups, range queries, composite filtering, sorting operations, and aggregations. Additionally, the study compares the scalability and efficiency of B-Tree versus Hash indexing against a non-indexed baseline and measures the storage cost necessary to maintain each index structure in both MySQL and MongoDB.

- **Literature Review and Analysis**

The selection of indexing strategies was grounded in a comprehensive literature review. B-Tree and hash indexing were found to be the most popular approaches in earlier research. Previous studies pointed out that there was a lack of recent research that compares indexed and non-indexed tables in typical transaction contexts. This gap informed the decision to compare indexing methods on MySQL and NoSQL environments.

- **Criteria for Comparison**

The indexing strategies were evaluated based on two primary quantitative metrics:

1. **Query Latency:** The speed is measured in seconds using the MySQL execution profiler. This metric captures the exact duration of query execution, excluding the network latency. For MongoDB, the Execution Time in Milliseconds metric from the execution stats command is observed.
2. **Storage Overhead:** The cost is measured in Kilobytes (KB). This metric measures the space efficiency of the index by querying the data length and index length from the information schema table system view.

- **Data Collection**

The SuperStore retail transaction dataset sourced from Kaggle was utilised for this assignment. The data was organised into a normalised schema to ensure relational integrity and appropriateness for indexing assessments. The raw data was first distributed across five normalised tables: Customers, Location, Products,

Orders, and Order Items. The primary keys were defined for each entity to ensure data uniqueness. Data cleaning involves cleaning composite attributes, removing redundant fields like 'Year', and generating surrogate keys for Customers and Locations. A baseline experimental table was constructed by joining the five tables, resulting in a final dataset of 51290 records. To ensure compatibility with the MySQL MEMORY engine used for hash indexing, columns with the text data type were explicitly altered to VARCHAR format prior to experimentation.

- **Experimental Setup**

The experiments were conducted using MySQL Community Server 8.0 and MySQL Workbench and MongoDB Server v8.2.3. The environment was strictly controlled to ensure consistent measurement. The SuperStore database was created to host all experimental tables. The MySQL profiling command was enabled to capture precise execution times. In MongoDB, the execution stats command was utilised to capture the time used for each query run. To support the memory-intensive Hash index in MySQL, the server's maximum heap table size was increased to 1GB to prevent table-full errors during data loading.

- **Implementation of Solutions**

Three respective distinct table environments for each platform were created to simulate the indexing scenarios.

Platform 1: MySQL

- **Scenario A (No Index):** the control table was created using the InnoDB engine. It contains the fully joined dataset including Order, Customer, Location, Product, and Sales information.
- **Scenario B (B-Tree Index):** A clone of the baseline data table was created using the InnoDB engine. Two B-Tree indexes were explicitly applied to test specific query types, specifically applied to the sales column to optimise range queries, and applied to the order ID column to optimise point lookups.
- **Scenario C (Hash Index):** A table was created using the MEMORY engine to support hash indexing. A hash index was explicitly applied to the order ID column to test O(1) lookup performance.

Platform 2: MongoDB (NoSQL)

- **Scenario D (No Index):** A standard collection scan was used as the control, where MongoDB traverses every document to find matches.
- **Scenario E (B-Tree Index):** Single field indexes on Order ID and Sales were created to support point and range queries respectively.

- Scenario F (Hash Index): A hash index was created and applied to Order ID to test lookup performance in a document store context.

• Evaluation and Comparison

To ensure reliability, a systematic testing protocol was followed involving repetitive execution. Firstly, five queries were executed against all three scenarios in each platform respectively to test different performance aspects. Point query was used to stimulate a specific transaction lookup, and range query was used to simulate a reporting query filtering for high-value transactions. Composite filter combines the exact match and range filter for a more advanced search. Sorting was done by sorting the dataset by Sales in descending order with limits. Aggregation groups the data by region and calculate the respective sum of sales. Each query type was executed exactly 5 times per scenario to account for cache warming and variance. After the execution cycles, the duration of each specific run was extracted and averaged.

• Validation of Results

The results were validated by verifying that the logic used to create the tables resulted in identical row counts and document counts across all relational and non-relational environments. Storage overhead was calculated dynamically using SQL math functions for MySQL, and the collStats command for MongoDB to provide precise Kilobyte (KB) measurements for the final report.

D. Tools and Technologies Used

MySQL Workbench 8.0 was used for relational database creation, SQL scripting, and execution profiling. MongoDB Server v8.2.3 and MongoDB Shell were used for the NoSQL implementation. Microsoft Excel was used for data logging and calculating average latencies. Structured Query Language (SQL) and MongoDB Query Language (MQL) were the core programming language used for defining schemas and querying data.

E. Flow Diagram or Architecture

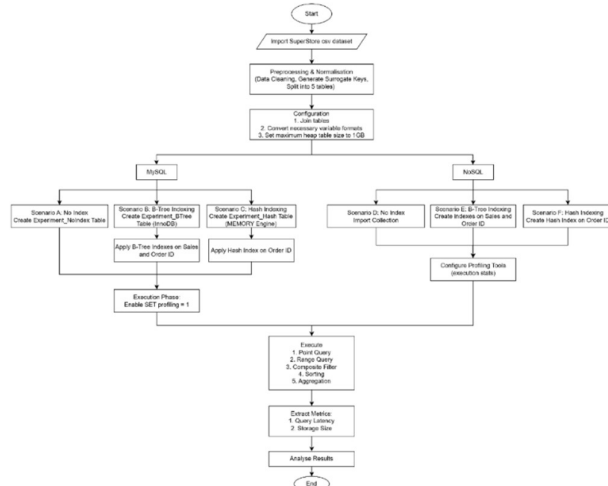


Fig. 1. Research Methodology Flowchart

This flowchart illustrates the sequence of data preprocessing, configurations, and execution steps used to evaluate the three indexing scenarios in MySQL and NoSQL environments.

F. Challenges and Limitations

A significant technical challenge encountered was the storage engine limitations associated with Hash Indexing. The MEMORY engine that was required for hash indexes in MySQL does not support the text data type and has a default size limit. To overcome this, the methodology involved two critical interventions, that is to convert text columns to varchar formats to ensure compatibility and increase the table size to 1GB to accommodate the full dataset in RAM.

Furthermore, ensuring experimental isolation in NoSQL environment presented a unique workflow challenge. Unlike the MySQL approach where distinct tables were created for each scenario, the MongoDB experiments were conducted on a single collection. Indexes have to be dropped between tests to prevent the query planner from using a pre-existing index, which would invalidate the index performance results.

G. Summary

This methodology creates a rigid framework for assessing database indexing performance across both relational (MySQL) and non-relational (MongoDB) systems. The study ensures the accuracy and reproducibility of performance measures by converting raw retail data into normalised relational and document structures. By isolating indexing algorithms within controlled environments like handling distinct storage engines in MySQL and index management in MongoDB, this approach directly addresses the research objectives. It provides empirical evidence to guide index selection for complex retrieval patterns including composite filtering, sorting, and aggregation operations.

IV. RESULTS AND DISCUSSION

A. Introduction to the Results and Discussion Section

This chapter presents the results that evaluate the three indexing methods, no index, hash index and B-tree index applied to the sales dataset across two database systems, MySQL and NoSQL database. These methods were performed on a normalized sales dataset using MySQL Workbench and MongoDB as well as the evaluation focused on the key performance metrics, including query execution time and storage overhead. Different types of queries, such as equality-based queries, ranged-based queries, composite, sorting and aggregation methods were used to measure the average execution time of different indexing methods. This section aims to identify the most suitable database and indexing strategy for retail transaction data by maximizing retrieval performance while minimizing storage overhead. Moreover, the strengths and limitations of each approach were listed in this chapter.

B. Presenting Results

• MySQL Query Execution Time Results

TABLE II. AVERAGE EXECUTION TIME FOR EQUALITY-BASED QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.150307
Hash Index	0.001294
B-tree Index	0.001376

Average Execution Time for Equality-Based Queries

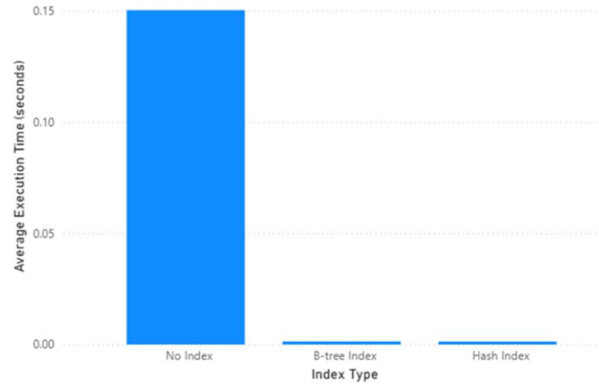


Fig. 2. Average Execution Time for Equality-Based Queries

The execution time of different query types was measured for each indexing strategy. Each query was executed multiple times and the average execution time was calculated to ensure a reliable comparison. Table II and Figure 2 shows the average execution time for equality-based queries across different indexing strategies in MySQL environment. The results indicate that hash indexing strategy achieves the lowest average execution time of 0.001294 seconds, whereas the table without indexing exhibits the highest average execution time of 0.150307 seconds. B-tree index has a faster average execution time than without index, where is slightly slower than hash index.

TABLE III. AVERAGE EXECUTION TIME FOR RANGE-BASED QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.026302
Hash Index	0.032723
B-tree Index	0.019909

Average Execution Time for Range-Based Queries

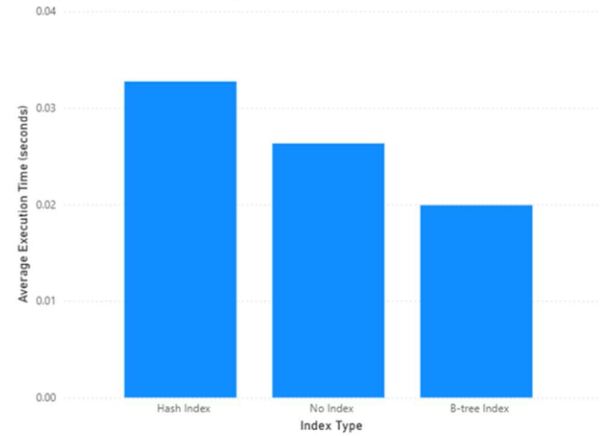


Fig 3. Average Execution Time for Range-Based Queries

Table III and Figure 3 show the comparison of average execution time for range-based queries across different indexing strategies in MySQL environment. The results indicate that the B-tree index has the lowest average execution time of 0.019909 seconds, whereas hash index has the highest average execution time of 0.032723 seconds. The table without indexing exhibits a higher average execution time than B-tree index, but a lower execution time than the hash index.

TABLE IV. AVERAGE EXECUTION TIME FOR COMPOSITE QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.154271
Hash Index	0.000694
B-tree Index	0.001011

Average Execution Time for Composite Queries

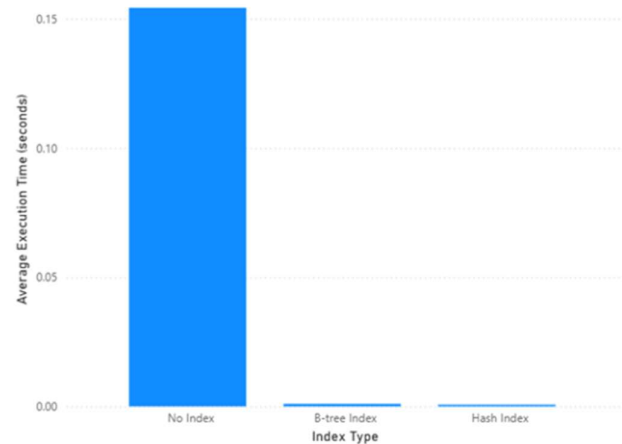


Fig. 4. Average Execution Time for Composite Queries

Table IV and Figure 4 illustrates the average execution time for composite queries across different indexing strategies in MySQL environment. The results indicate that the hash

index has the lowest average execution time of 0.000694 seconds, whereas the table without index has the highest average execution time of 0.154271 seconds. The B-tree index demonstrates a lower average execution time than the table without indexing, but is slightly higher than the hash index.

TABLE V. AVERAGE EXECUTION TIME FOR SORTING QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.207959
Hash Index	0.236618
B-tree Index	0.001816

Average Execution Time for Sorting Queries

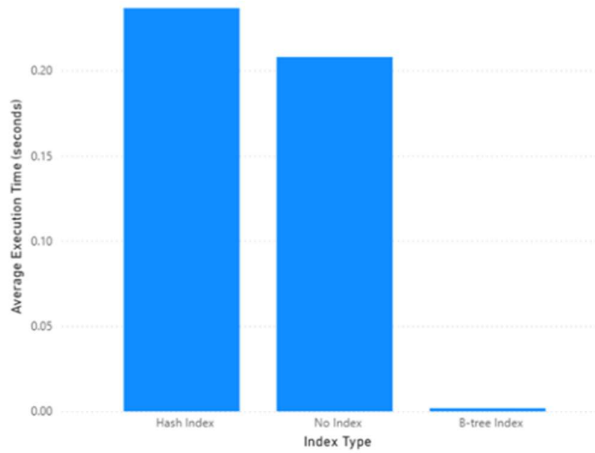


Fig. 5. Average Execution Time for Sorting Queries

Table V and Figure 5 illustrates the average execution time for sorting queries across different indexing strategies in MySQL environment. The results indicate that the B-tree index has the lowest average execution time of 0.001816 seconds, whereas hash index has the highest average execution time of 0.236618 seconds. The table without indexing demonstrates a slightly lower average execution time than hash index, but a higher execution time than the B-tree index.

TABLE VI. AVERAGE EXECUTION TIME FOR AGGREGATION QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.206772
Hash Index	0.201838
B-tree Index	0.139689

Average Execution Time for Aggregation Queries

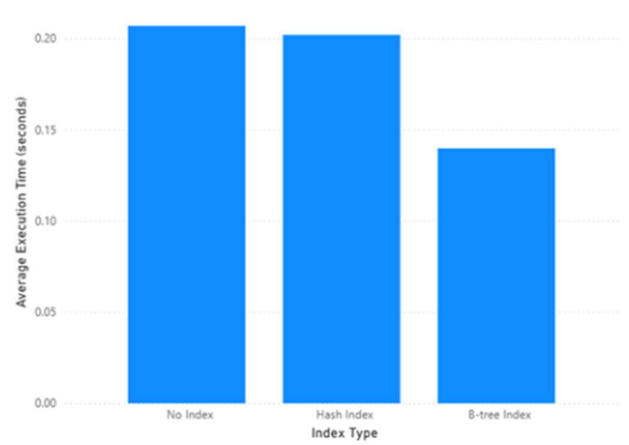


Fig. 6. Average Execution Time for Aggregation Queries

Table VI and Figure 6 illustrate the average execution time for aggregation queries across different indexing strategies in MySQL environment. The results indicate that the B-tree index has the lowest average execution time of 0.139689 seconds, whereas the table without indexing has the highest average execution time of 0.206772 seconds. The hash index exhibits a higher average execution time than the B-tree index, but a slightly lower execution time than the table without indexing.

MySQL Storage Overhead Results

TABLE VII. STORAGE OVERHEAD OF INDEXING TECHNIQUES

Index Type	Data Size (KB)	Index Size (KB)	Total Size (KB)
No Index	11792	0	11792
Hash Index	347010.2	1503.34	348513.55
B-tree Index	16	0	16

Storage Overhead of Indexing Techniques

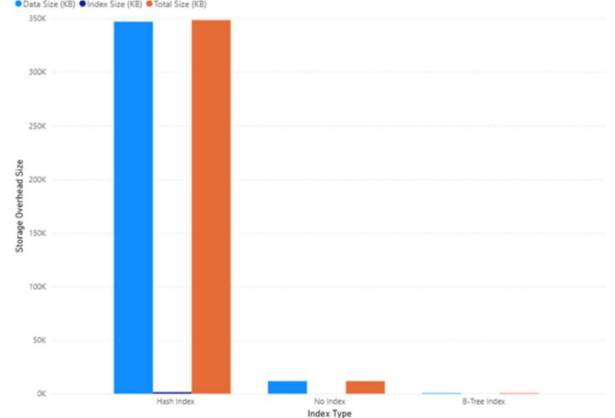


Fig. 7. Storage Overhead of Indexing Techniques

Table VII and Figure 7 presents the storage overhead results across different indexing

techniques in MySQL environment, including data size and index size. In terms of data size, the hash index has the largest storage size at 347010.2 KB, followed by the table without indexing with 11792 KB. The B-tree index records the lowest data storage size at 16KB. In terms of index size, the hash index has the largest index size of 1503.34 KB, whereas both the B-tree index and the table without indexing have an index size of 0 KB. Overall, in terms of total storage overhead, the B-tree index has the lowest storage size at 16KB, followed by the table without indexing with a total size of 11792 KB. The hash index exhibits the highest total storage size of 348513.55 KB.

- **NoSQL Query Execution Time Results**

TABLE VIII. AVERAGE EXECUTION TIME FOR EQUALITY-BASED QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.058
Hash Index	0.004
B-tree Index	0.0002

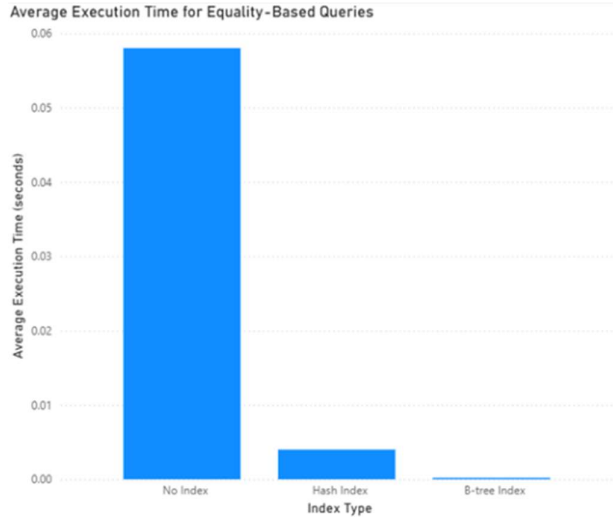


Fig. 8. Average Execution Time for Equality-Based Queries

Table VIII and Figure 8 illustrate the average execution time for equality-based queries across different indexing strategies in a NoSQL environment. The results show that the B-tree indexing strategy has the lowest average execution time of 0.0002 seconds, whereas the table without indexing achieves the highest average execution time of 0.058 seconds. The B-tree index shows faster average execution time as compared to the hash indexing strategy.

TABLE IX. AVERAGE EXECUTION TIME FOR RANGE-BASED QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.0362
Hash Index	0.0294
B-tree Index	0.0188

No Index	0.0362
Hash Index	0.0294
B-tree Index	0.0188

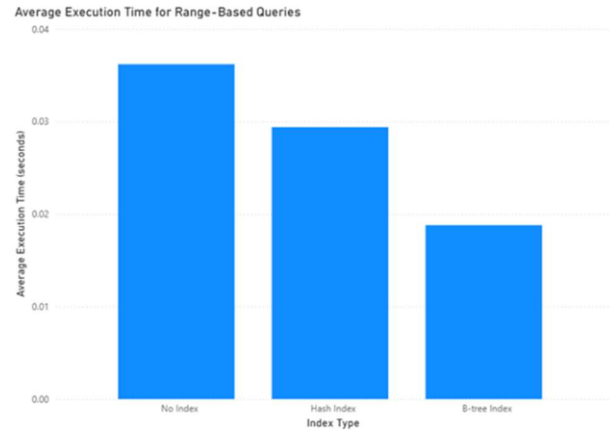


Fig. 9. Average Execution Time for Range-Based Queries

Table IX and Figure 9 demonstrate the average execution time for range-based queries across different indexing strategies in a NoSQL environment. The results show that the B-tree indexing strategy has the lowest average execution time of 0.0188 seconds, whereas the table without indexing achieves the highest average execution time of 0.0362 seconds. The B-tree indexing strategy outperforms the hash indexing strategy in terms of average execution time.

TABLE X. AVERAGE EXECUTION TIME FOR COMPOSITE QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.073
Hash Index	0
B-tree Index	0.001

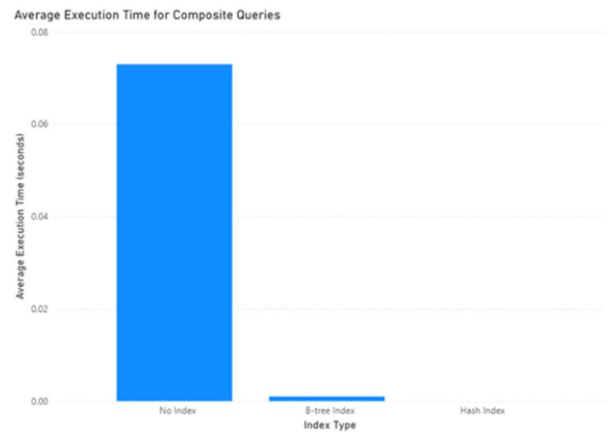


Fig. 10. Average Execution Time for Composite Queries

Table X and Figure 10 illustrate the average execution time for composite queries across

different indexing strategies in a NoSQL environment. The results indicate that the Hash indexing strategy has the lowest average execution time of 0 seconds, whereas the table without indexing achieves the highest average execution time of 0.073 seconds. The hash indexing strategy outperforms the B-tree indexing strategy in terms of average execution time.

TABLE XI. AVERAGE EXECUTION TIME FOR SORTING QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.0478
Hash Index	0
B-tree Index	0

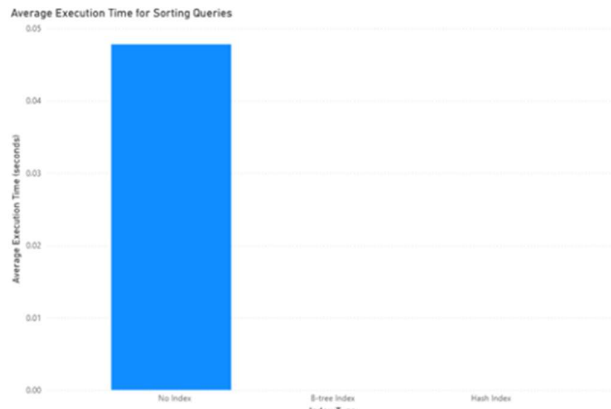


Fig. 11. Average Execution Time for Sorting Queries

Table XI and Figure 11 show the average execution time for sorting queries across different indexing strategies in a NoSQL environment. The results indicate that both the B-tree and hash indexing strategies achieve the lowest average execution time of 0 seconds, whereas the table without indexing has the highest average execution time of 0.0478 seconds.

TABLE XII. AVERAGE EXECUTION TIME FOR AGGREGATION QUERIES

Index Type	Average Execution Time (seconds)
No Index	0.0424
Hash Index	0.0004
B-tree Index	0.043

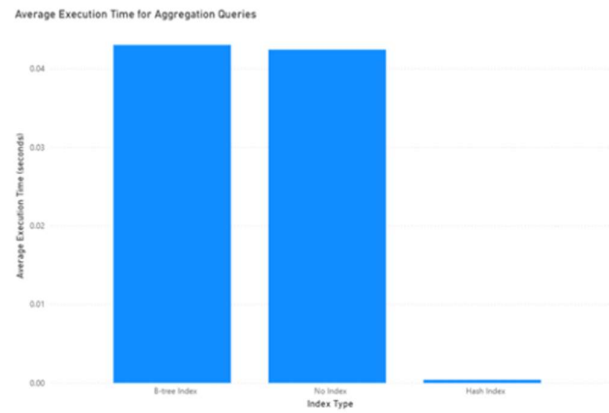


Fig. 12. Average Execution Time for Aggregation Queries

Both Table XII and Figure 12 show the average execution time for aggregation queries across different indexing strategies in a NoSQL environment. The results indicate that the hash indexing strategy achieves the lowest average execution time of 0.0004 seconds, whereas B-tree index achieves the highest average execution time of 0.043 seconds. The table without indexing achieves a slightly lower average execution time of 0.0424 seconds compared to B-tree index.

• NoSQL Storage Overhead Results

TABLE XIII. STORAGE OVERHEAD OF INDEXING TECHNIQUES

Index Type	Data Size (KB)	Index Size (KB)	Total Size (KB)
No Index	7016	516	7532
Hash Index	7016	980	7996
B-tree Index	7016	876	7893

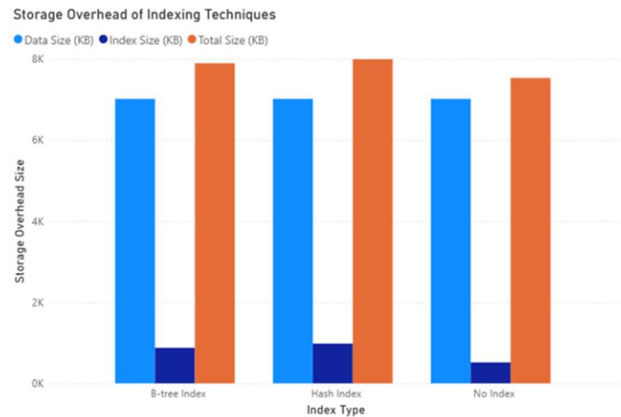


Fig. 13. Storage Overhead of Indexing Techniques

Table XIII and Figure 13 shows the storage overhead results across different indexing techniques in a NoSQL environment. The data size is consistent across all indexing techniques, with a value of 7016 KB. In terms of index size, the hash index has the largest index size of 980 KB,

whereas the table without indexing has the smallest index size of 516 KB. Overall, in terms of total storage overhead, the table without indexing has the least storage size at 7532 KB, followed by the B-tree index with a total size of 7893 KB. The hash index has the highest total storage size at 7996 KB.

C. Comparative Analysis and Discussion

The presented results show a significant reduction in query latency when hash and B-tree indexing are applied compared to the no-index configuration, in both MySQL and NoSQL environments. This is because tables without indexes require $O(N)$ full scans to locate matching records by examining every single entry sequentially from the start until it finds a match. This results in a longer data retrieval time, especially when the database grows larger by N , it slows down the process.

However, the hash index achieves the lowest latency for equality and composite queries in MySQL as it maps the search key directly to the location where the record is stored, which achieves a time complexity of $O(1)$ [3]. In NoSQL, the hash index shows an average execution time of 0 seconds for composite and sorting queries. This is because documents in MongoDB are viewed as sets instead of ordered lists, and hash index randomizes the order of the data, which explains why hash indexes can't support sorting. For composite queries in NoSQL, MongoDB does not allow hash indexes on multiple fields, as hash indexes cannot be combined with other index types in a single index. Multiple conditions in a composite query may be satisfied by different array elements rather than a single logical entity. The hash index also achieves the lowest execution time for aggregation queries in MongoDB, as it allows $O(1)$ data access by mapping values directly to memory addresses without the need to perform sorting operations [23]. B-tree indexing shows consistently low latency across different types of queries in MySQL, especially in range, sorting and aggregation queries. This is because it utilizes a hierarchical structure which allows fast logarithmic time complexity ($O(\log N)$) during data retrieval in both NoSQL and MySQL environments [24]. In terms of NoSQL environment, B-tree indexing performs better in equality, range and sorting queries.

Furthermore, table 6 shows that the B-tree index has the lowest storage overhead, whereas the hash index consumes the most storage space in MySQL. This is because hash indexing uses a bucket-based structure which allocates extra space to manage collisions when multiple keys map to the same bucket. Hash indexes often reserve additional space and may expand their internal structures when insertion capacity is reached. This results in increased storage consumption even when the actual data volume does not increase proportionally [25]. In comparison, B-tree index has the lowest storage overhead as it utilizes space efficiently within tree nodes, where keys are stored in a balanced hierarchical structure that allows nodes to remain densely packed, thereby reducing unused space. Instead of reserving large unused areas, B-tree nodes are dynamically allocated and grow incrementally as data is inserted, which enables efficient storage utilization while maintaining balanced search performance [26]. In terms of NoSQL based on Table 12, the table without indexing consumes the least storage overhead, as the collection only stores the raw

data itself. When B-tree or hash index is added, an additional data structure is created to store the index, which consumes extra disk space.

In short, each indexing method exhibits unique strengths and weaknesses when evaluated based on execution time and storage overhead in both MySQL and NoSQL environments. In MySQL, the weaknesses of the hash index include consuming the most storage due to bucket allocation and collision handling, as well as poor performance in range, sorting or aggregation queries. However, the strengths of hash index include utilizing hash functions for direct key-based lookup and achieving the lowest execution time for equality-based and composite queries. In NoSQL databases such as MongoDB, the hash index performs effectively for aggregation queries, but shows limitations for other query types. Similarly, hash indexing in NoSQL consumes the most storage overhead.

In comparison, in MySQL environment, the weaknesses of B-tree index include higher execution time for equality-based and composite queries as well as additional index maintenance overhead during insert and update operations. The strengths of B-tree index include low storage overhead, consistent execution performance across different types of queries and an ordered structure that enables the system to perform efficient sequential scanning and grouping. In NoSQL environment, B-tree indexing performs well in equality, range and sorting queries, as well as consumes a lower storage overhead compared to hash indexing.

D. Overall Discussion

This study evaluated the performance of three scenarios in both MySQL and NoSQL environments, which are tables with no indexing, hash indexing and B-tree indexing to identify the significant impact of indexing on query performance and storage efficiency. Besides, the study aimed to identify the most optimal indexing method to be applied in different databases for daily retail transaction operations. In both MySQL and NoSQL environments, the findings show that the B-tree index has the most optimal overall performance as it provides an ordered tree structure, the lowest storage overhead and low execution time when retrieving data across different types of queries, especially for range-based, sorting and aggregation queries in MySQL, whereas equality, range and sorting queries in NoSQL. In comparison, although hash index has the lowest execution time for equality-based queries, it consumes the highest amount of storage due to bucket allocation and collision handling, which makes it less optimal than the B-tree index.

One of the practical implications suggested by the findings of this study is that database designers and system administrators can utilize the B-tree indexing in retail store datasets for both MySQL and NoSQL environments, as it is suitable in operations involving the retrieval of sales records within a specific data ranges and maintains a consistent time complexity of $O(\log N)$ across different query types. This speeds up data analysts' productivity by enabling analytical sales reports to be generated in minimal time. Besides, B-tree indexing is suggested for business applications as it consumes the lowest total storage space, to reduce the monetary costs related to storage hardware and cloud hosting. This study also suggests that B-tree indexing can be

utilized to minimize the CPU usage during query execution, allowing businesses to maximize hardware performance without the need for frequent server upgrades to achieve high performance.

In this study, there are some limitations exist and need to be addressed in future research. Firstly, this study is inconsistent in terms of engine and storage usage across different environments. In MySQL, the InnoDB engine is used for the table without indexing and the B-tree indexing scenarios, where hash indexing uses MEMORY engine because MySQL requires in-memory storage for hash indexes. In NoSQL, MongoDB uses the WiredTiger engine for all scenarios, but its internal caching mechanism may have kept the dataset entirely in RAM. As a result, the findings of this study may be influenced by the performance differences between RAM based or disk-based storage rather than by the indexing algorithms themselves especially in MySQL. Future studies should adopt the same type of storage engine for hash indexing, B-tree indexing or other indexing methods to achieve more consistent performance comparisons. Secondly, the indexing methods were performed on a dataset containing 51290 records, which is considered small by modern database standards. In MongoDB, a dataset of this scale is often small enough to fit entirely in the memory, which may hide the real disk I/O overhead involved in B-tree traversals. Hence, future studies should focus on larger datasets with millions or billions of records to evaluate whether the B-tree indexing remains stable in terms of execution speed and storage overhead.

Furthermore, this study focused only on the metrics of average query execution time and storage overhead, while neglecting other factors that may influence the suitability of indexing strategies under various circumstances, such as index maintenance costs or real-time workload variability. Therefore, in order to understand the comprehend the computational cost of maintaining the index, future studies should take into account additional metrics like CPU utilization during hash table resizing and assess index performance under mixed operations like 80% scans and 20% inserts.

E. Visual Representation of Results

TABLE XIV. AVERAGE EXECUTION TIME (SECONDS) OF QUERY TYPES ACROSS DIFFERENT INDEXING STRATEGIES (MySQL)

Index Type	Equality	Range	Composite	Sorting	Aggregation
No Index	0.150307	0.026302	0.154271	0.207959	0.206772
Hash Index	0.001294	0.032723	0.000694	0.236618	0.201838
B-tree Index	0.001376	0.019909	0.001011	0.001816	0.139686

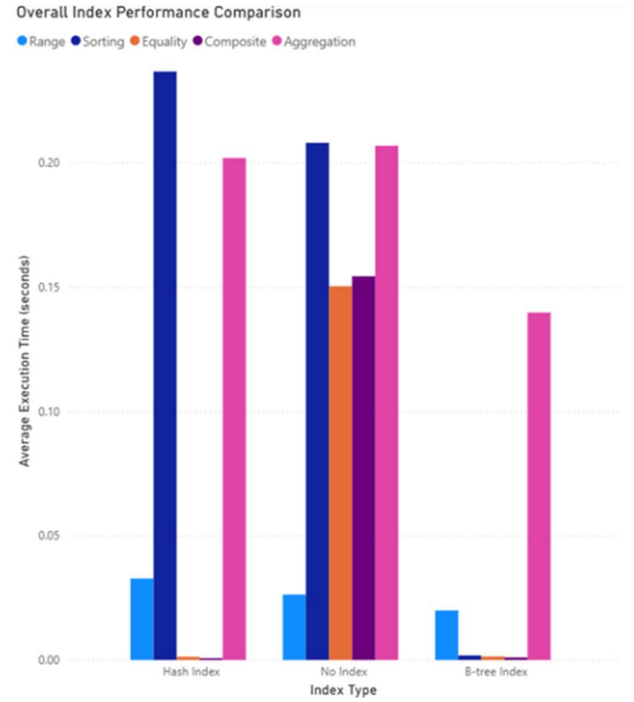


Fig. 14. Average Execution Time (seconds) of Query Types Across Different Indexing Strategies (MySQL)

Table XIV and Figure 14 show the comparison of average execution times for different query types across various indexing strategies in a MySQL environment. The results indicate that the table without indexing exhibits a consistently higher average execution time across all query types compared to the indexed tables. The B-tree index shows a lower average execution time for all query types, particularly for sorting, equality and composite queries. In short, the results suggest that the B-tree index outperforms both the hash index and the table without indexing in terms of average execution time.

TABLE XV. AVERAGE EXECUTION TIME (SECONDS) OF QUERY TYPES ACROSS DIFFERENT INDEXING STRATEGIES (No SQL)

Index Type	Equality	Range	Composite	Sorting	Aggregation
No Index	0.058	0.0362	0.073	0.0478	0.0424
Hash Index	0.004	0.0294	0	0	0.0004
B-tree Index	0.0002	0.0188	0.001	0	0.043

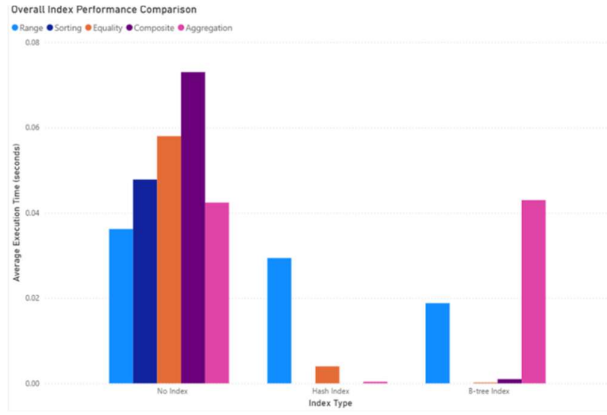


Fig. 15. Average Execution Time (seconds) of Query Types Across Different Indexing Strategies (No SQL)

Table XV and Figure 15 show the comparison of average execution times for different query types across various indexing strategies in a NoSQL environment. The results show that the table without indexing consistently exhibits higher average execution times across all query types compared to the indexed tables. The B-tree index shows lower average execution times for most query types, except for aggregation queries. In short, the results suggest that the B-tree index outperforms both the hash index and table without indexing.

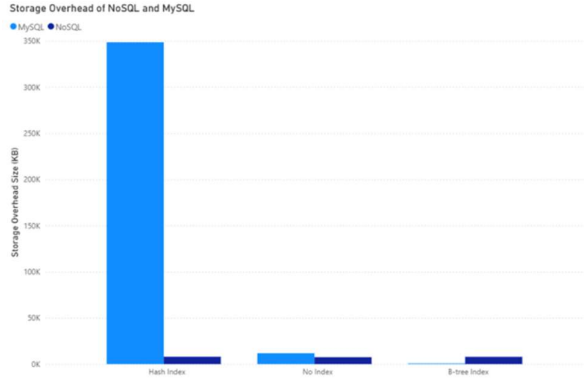


Fig. 16. Storage Overhead Comparison between MySQL and NoSQL

Figure 16 shows the comparison of storage overhead between MySQL and NoSQL environments. The results indicate that the hash index has the highest storage overhead among the indexing methods in both MySQL and NoSQL. The table without indexing shows the lowest storage overhead in the NoSQL environment, but its storage overhead is higher than that of the B-tree index in the MySQL environment. The B-tree index has a higher storage overhead than table without indexing in the NoSQL environment, but it achieves the lowest storage overhead among the indexing techniques in MySQL. Overall, most indexing methods in MySQL incur higher storage overhead compared to NoSQL.

F. Critical Evaluation

The objective of this study was to determine the most suitable indexing strategy by implementing and configuring different types of indexing scenarios on a dataset, specifically no index, B-tree index and hash indexing, within both MySQL and MongoDB environments. The

findings show that indexing increases the data retrieval speed compared to a table without indexing. This is because non-indexed tables in MySQL and collections in MongoDB perform $O(N)$ complete table scans, which result in higher average execution times than indexed tables. B-tree indexing demonstrated consistently low execution times across different query types while maintaining minimal storage overhead due to its balanced hierarchical structure. Therefore, B-tree indexing is the most suitable indexing solution in both environments for retail transaction databases, as it effectively maximizes retrieval performance while minimizing storage costs of the system.

Moreover, the second objective was to measure and analyze the data retrieval speeds for point lookup, range, composite, sorting and aggregation queries in both MySQL and NoSQL environments under various configurations. The results demonstrate that hash index uses hash functions to map search keys directly to memory addresses with a time complexity of $O(1)$, which achieves the lowest average execution time for equality-based and composite queries in MySQL and for aggregation queries in NoSQL. In comparison, the B-tree index achieves the fastest execution time for sorting, range and aggregation queries in MySQL as well as for equality, range and sorting queries in NoSQL because of its ordered tree structure, which allows lead nodes to be traversed sequentially without the need for performing additional sorting operations. The findings confirmed that instead of providing a universally optimal performance, the retrieval speed of indexing methods varies based on different type of query.

The final objective of the study was to assess the data storage cost of different indexing methods in both NoSQL and MySQL environments through calculating the data space and index size consumed by each indexing techniques. The results of the disk space analysis show that the hash index consumes the highest amount of storage compared to both the no index table and the B-tree index in MySQL environment, which is due to bucket allocation and collision handling. However, B-tree indexing shows minimal storage overhead, making it more suitable for managing large-scale retail databases where storage efficiency is an important concern. In NoSQL environment, tables without indexing have the lowest storage overhead compared to the other indexing methods. The results also show that the B-tree index consumes less storage overhead compared to hash indexing.

G. Linking to Literature Review

This section relates the findings of the study to the existing literature reviewed in Chapter 3 to evaluate whether the findings of this study are consistent with or contradict previous studies on database indexing techniques. Firstly, the results of this study indicate that hash indexing performs well for equality-based and composite queries in MySQL environment, as well as aggregation in NoSQL environment. This outcome is consistent with the study by Nakazono et al. [3], which mentioned that hash indexing performs well in point-heavy workloads due to its constant time complexity $O(1)$, but has limitations when handling range queries. In the MongoDB environment, the hash index was not utilized for range-based aggregation queries, which results in less efficient collection of scans. Moreover, this study found that B-tree indexing demonstrates consistent execution

performance, especially for range-based, sorting and aggregation queries in MySQL environment as well as equality, composite and sorting queries in NoSQL environment. This aligns to the study by Aminuddin et al. [10], which mentioned that the B-tree indexing technique is efficient in balancing the data and enhancing the query processing at a logarithmic time complexity of $O(\log N)$. The study by Li et al. [16] also supports these findings, as it mentioned that B-tree indexes remain stable and reliable across different types of workloads. Furthermore, the results of this study indicate that hash indexing has the highest storage overhead in terms of data size for both MySQL and NoSQL environments, which aligns with the study by Tao et al. [11], who mentioned that traditional hash indexes require increased storage due to collision overhead and limited adaptability. In the NoSQL context, hashed values are randomized, which prevents WiredTiger's prefix compression from working effectively. Hence, hash indexes consume more storage than B-tree indexes. Lastly, this study found that B-tree indexing has the lowest storage overhead as compared to hash index in both MySQL and NoSQL environments. However, this observation does not fully align with Lee et al. [21] who reported that poor B-tree utilization can degrade storage efficiency. The difference is this study focuses on evaluating index storage size using standard InnoDB and WiredTiger configurations, whereas Lee et al. [21] examined node-level space efficiency in flash-based storage systems.

H. Conclude the Results and Discussion

In conclusion, the results indicate that indexing improves the query execution performance as compared to tables without indexing. In terms of storage overhead, tables without indexes perform better as compared to those with indexing. The B-tree index was found to be the most effective overall indexing technique, as it offers low storage overhead and low average execution time across different queries in both MySQL and NoSQL environments. The low storage overhead of the B-tree index is an advantage, as it reduces the cost of running large-scale retail databases. For Hash index, although it achieves fastest execution time for equality-based and composite queries due to its direct key-based lookup, it still consumes the highest amount of storage space. Hence, the results confirm that B-tree indexing is more suitable than hash indexing as a method to be applied in daily retail transaction database operations.

V. CONCLUSION AND FUTURE WORK

This study analyzed the performance, storage, and return time of different indexing techniques based on a controlled retail sales dataset environment. After investigating the observed result, the non-indexed access has the worst performance compared to hash and B-tree indexes, since it requires a full table scan process to process queries. Besides, the hash index has the best overall performance on exact match queries, but low performance on range or ordered query types. Meanwhile, although the B-Tree indexing methods may incur additional cost, they provide the most rounded and fully featured capability to retrieval-type requests.

These findings achieved the objective that this study aimed for under the introduction, which is to evaluate and

compare different indexing techniques through analyzing the way that they retrieved information, produced scalability, and storage cost. Overall, the result states that hash indexing provides efficiency for particular types of lookups, while the B-Tree indexed will be more suitable for general database application workloads of mixed query types. Besides, the results of the non-indexed public database also show the importance of indexing to provide efficiency on scaling while retrieving data.

Findings from this paper also provide practical ramifications for design practice and are an important resource for database practitioners. It also points out the suitable indexed techniques based on different scenarios and requirements. For example, B-Tree indexing has better performance and is more suitable for range queries and sorting operations, and large amounts of analytical workloads. Meanwhile, hash indexing is better for fast direct searches for limited or no access systems to persistent storage.

On the other hand, there are various limitations in this study. For example, the comparison of performance and storage cost between different table types may be affected due to the adoption of the MEMORY storage engine in hash indexing and the utilization of InnoDB for B-Tree and non-indexed tables. Besides, the dataset used in this study is only from one application domain, which may also cause limitations on generalization.

Last but not least, the future research and work for this study may include the investigation of hybrid or hybrid adaptive indexing techniques to assess and evaluate the performance and storage engines of them on larger or real-time dataset sources. Therefore, this paper has provided important insights into the database indexing trade-off. It also provides and supports the design and optimization area of a database management system (DBMS).

Github:

https://github.com/isabelleyaul/Group31_Database_FinalAssessment_code

REFERENCES

- [1] TiDB, "Understanding B-Tree and Hash Indexing in Databases." Accessed: Dec. 21, 2025. [Online]. Available: Understanding B-Tree and Hash Indexing in Databases
- [2] M. Basalla, J. Schneider, M. Luksik, R. Jaakonmäki, and J. Vom Brocke, "On Latency of E-Commerce Platforms," *Journal of Organizational Computing and Electronic Commerce*, vol. 31, no. 1, pp. 1–17, 2021, doi: 10.1080/10919392.2021.1882240.
- [3] S. Nakazono, H. Kawashima, Y. Bessho, and T. Nakamori, "Griffin: Fast Transactional Database Index with Hash and B+-Tree," *arXiv:2407.13294 [cs.DB]*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.13294>
- [4] L. Ondráček and O. Mička, "Cache-Oblivious Representation of B-Tree Structures," *arXiv:2209.09166 [cs.DS]*, 2024. [Online]. Available: <https://arxiv.org/abs/2209.09166>
- [5] H. An, F. Wang, D. Feng, X. Zou, Z. Liu, and J. Zhang, "Marlin: A Concurrent and Write-Optimized B+-tree Index on Disaggregated Memory," in **Proc. 52nd Int. Conf. Parallel Process. (ICPP)**, Salt Lake City, UT, USA, Aug. 2023, pp. 695–704, doi: 10.1145/3605573.3605576.
- [6] X. Chang and W. Ma, "Research on Embedded Database Indexing Technology in Avionics Systems," in *2024 3rd Int. Conf. on Cloud Computing, Big Data Application and Software Engineering (CBASE)*, 2024, pp. 221–225, doi: 10.1109/CBASE64041.2024.10824251.

[7] M. Al Khaldy, A. Shaheen, W. Alzyadat, and A. Alhroob, “An Efficient Primary Indexing Method with Sibling Pointers for Large-Scale Database Systems,” **Eng. Technol. Appl. Sci. Res.**, vol. 15, no. 4, pp. 24691–24697, Aug. 2025, doi: 10.48084/etasr.11575.

[8] I. Pujiono, E. H. Rachmawanto, and N. A. S. Winarsih, “Analysis and Comparison of Binary and Interpolation Search Algorithms in a B-tree,” in **Proc. 25th Pan-Hellenic Conf. Informatics (PCI ’21)**, 2022, pp. 74–78, doi: 10.1145/3503823.3503837.

[9] J. Schmeißer, C. Lutz, and V. Markl, “Efficiently Indexing Large Data on GPUs with Fast Interconnects,” in **Proc. 28th Int. Conf. Extending Database Technology (EDBT)**, Mar. 2025, pp. 661–667, doi:10.48786/edbt.2025.53.

[10] A. Aminuddin, M. Z. Saringat, S. A. Mostofa, A. Mustapha, and M. H. Hassan, “A Case Study on B-Tree Database Indexing Technique,” **Journal of Software, Computing and Data Management (JScDM)**, vol. 1, no. 1, pp. 1–10, Mar. 2020. doi: 10.30880/jsedm.2020.01.01.004. [Online]. Available: <https://publisher.uthm.edu.my/ojs/index.php/jsedm/article/view/6828>

[11] Y. Tao, X. Zhang, and H. Li, “Pea Hash: A Performant Extendible Adaptive Hashing Index,” in **Proc. ACM SIGMOD International Conference on Management of Data**, 2023, pp. 1–25, doi: 10.1145/3588962.

[12] J. Jiang, D. He, Y. Hu, D. Liu, C. Xiao, H. Bi, Y. Zhang, C. Jiang, and Z. Fu, “CompassDB: Pioneering High-Performance Key-Value Store with Perfect Hash,” *arXiv:2406.18099v1 [cs.DB]*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.18099>

[13] Q. Zhang, Y. Jia, F. Li, and L. Fan, “An Efficient Encrypted Speech Retrieval Based on Unsupervised Hashing and B+ Tree Dynamic Index,” **Computers, Materials & Continua**, vol. 76, no. 1, pp. 107–128, 2023. doi: 10.32604/cmc.2023.038911.

[14] X. Li, Y. Chen, and H. Wang, “Block-Level Data Deduplication on User-Mode Virtual Block Device with Hash and B+ Tree Index,” in *2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*, Changchun, China, Feb. 24–26, 2023, pp. 887–894, doi: 10.1109/EEBDA56825.2023.10090509.

[15] M. Altahat, Z. Han, and Y. Zhang, “Hybrid B-Tree and Enhanced Merkle Hash Tree System for Secure Cloud Data Management,” in *Proc. Int. Conf. Visual Analytics Data Visualization (ICVADV)*, 2025, pp. 187–192, doi: 10.1109/ICVADV63329.2025.10961342.

[16] X. Li, A. Jindal, and S. Madden, “Updatable learned indexes meet disk-resident DBMS – From evaluations to design choices,” *arXiv*, May 2023. [Online]. Available: <https://arxiv.org/ftp/arxiv/papers/2305/2305.01237.pdf>

[17] H. Cha, X. Hao, T. Wang, H. Zhang, A. Akella, and X. Yu, “Blink-hash: An Adaptive Hybrid Index for In-Memory Time-Series Databases,” *Proc. VLDB Endow.*, vol. 16, no. 6, pp. 1235–1248, 2023, doi: 10.14778/3583140.3583143.

[18] W. Lyu and G. Graefe, “Indexing Join Inputs for Fast Queries and Maintenance,” *arXiv:2502.10874 [cs.DB]*, Feb. 2025. [Online]. Available: <https://arxiv.org/pdf/2502.10874>

[19] S. Wu, X. Yu, X. Feng, F. Li, W. Cao, and G. Chen, “Progressive Neural Index Search for Database System,” *arXiv:1912.07001 [cs.DB]*, 2019. [Online]. Available: <https://arxiv.org/ftp/arxiv/papers/1912/1912.07001.pdf>

[20] C. Gao, S. Ballijepalli, and J. Wang, “Revisiting B-tree Compression: An Experimental Study,” **Proc. ACM on Management of Data**, vol. 2, no. 3, pp. 1–25, 2024, doi: 10.1145/3654972.

[21] B.-H. Lee, M. An, and S.-W. Lee, “A case for space compaction of B-tree nodes on flash storage,” *IEEE Access*, vol. 11, pp. 38149–38155, Apr. 2023, doi: 10.1109/ACCESS.2023.3267084.

[22] H. Lan, Z. Bao, J. S. Culpepper and R. Borovica-Gajic, “Updatable Learned Indexes Meet Disk-Resident DBMS – From Evaluations to Design Choices,” **Proc. ACM Manag. Data**, vol. 1, no. 2, Art. 139, 2023, doi:10.1145/3589284.

[23] E. Botoeva, D. Calvanese, B. Cogrel, and G. Xiao, “Expressivity and complexity of MongoDB queries,” in *Proc. 21st Int. Conf. Database Theory (ICDT 2018)*, 2018, pp. 9:1–9:19.

[24] A. Kaushik, T. Vinay, D. Mishra, and A. U. Patil, “Optimizing spatial queries in NoSQL databases: A comparative analysis of B-tree, hashed, and geospatial indexing techniques,” in *Proceedings of the 2024*

International Conference on Knowledge Engineering and Communication Systems (ICKECS), vol. 1, Apr. 2024, pp. 1–6.

[25] B. Zhang, S. Zheng, L. Nie, Z. Qi, L. Huang, and H. Mei, “Exploiting persistent CPU cache for scalable persistent hash index,” in *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*, May 2024, pp. 3851–3864.

[26] B. K. Kim, G. W. Kim, and D. H. Lee, “A novel B-tree index with cascade memory nodes for improving sequential write performance on flash storage devices,” *Applied Sciences*, vol. 10, no. 3, Art. no. 747, 2020.

CONTRIBUTION TABLE

Student Name (ID)	Percentage of the Overall Participation
Isabelle Yau Li Shuang (23021165)	100% out of 100%
Leow Yan Yi (23020324)	100% out of 100%
Peggy Liow En Ki (23020852)	100% out of 100%